



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE

ENERO - JUNIO 2026

CARRERA

Ingeniería en Sistemas computacionales

MATERIA

Patrones de Diseño

TÍTULO

Proyecto Final

NOMBRES Y NÚMEROS DE CONTROL DE ALUMNOS

Gayosso Pérez Karla Yuray - 22210764

Nolasco Vazquez José Antonio - 22210326

San Roman Castillo Gabriel - 19211731

León Cruz Jorge Joshel - 22210772

NOMBRE DEL MAESTRO

MARIBEL GUERRERO LUIS

FECHA DE ENTREGA

29 de Mayo del 2026

Instrucciones

Seleccionará uno de los proyectos recibidos en equipo (deberá incluir un patrón por categoría), en caso de proponer un nuevo proyecto tendrá que ser valorado y aprobado a más tardar el día de 11 de mayo.

Seleccione un patrón por unidad para incorporar a su proyecto:

- ➔ Patrones Creacionales
- ➔ Patrones Estructura
- ➔ Patrones de Comportamiento
- ➔ Patrones de Arquitectura

¿Qué son los patrones creacionales?

Son patrones de diseño de software que se enfocan en la forma en que se crean los objetos dentro de un programa, su objetivo es hacer que la creación de objetos sea más flexible, reutilizable y fácil de mantener

¿Para qué sirve?

- Reduce el acoplamiento entre clases
- Reutiliza el código
- Facilita el mantenimiento
- Controla el ¿Cómo? y ¿Cuándo? se crean los objetos

Tipos de patrones creacionales

- Singleton
- Factory Method
- Abstract Factory
- Builder
- Prototype

Ventajas

- Mejor organización del código
- Menor acoplamiento
- Mayor reutilización
- Facilitan mantenimiento y escalabilidad
- Mejor control de creación de objetos

Desventajas

- Aumentan la complejidad del sistema
- Más clases y estructuras
- Pueden ser innecesarios en proyectos pequeños
- Requieren experiencia para implementarlos correctamente

¿Qué son los patrones estructurales?

Son patrones de diseño de software que se enfocan en la forma en que las clases y objetos se organizan para formar estructuras más grandes y eficientes, su objetivo es facilitar la relación entre componentes y mejorar la flexibilidad del sistema.

¿Para qué sirven?

- Permiten reutilizar estructuras existentes
- Facilitan la integración entre clases y objetos
- Reducen el acoplamiento
- Mejoran la organización del sistema
- Simplifican sistemas complejos

Tipos de patrones estructurales

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Ventajas

- Mayor reutilización de código
- Mejor organización de componentes
- Facilitan la escalabilidad
- Permiten integrar sistemas incompatibles
- Mayor flexibilidad en el diseño

Desventajas

- Incrementan la complejidad del sistema
- Pueden generar muchas clases adicionales
- Algunos patrones son difíciles de comprender
- Mayor tiempo de implementación

¿Qué son los patrones de comportamiento?

Son patrones de diseño de software que se enfocan en la comunicación e interacción entre objetos, su objetivo es distribuir responsabilidades y definir cómo colaboran los objetos dentro del sistema.

¿Para qué sirven?

- Mejoran la comunicación entre objetos
- Reducen el acoplamiento
- Facilitan la distribución de responsabilidades
- Permiten cambiar comportamientos dinámicamente
- Hacen el código más flexible y mantenible

Tipos de patrones de comportamiento

- Observer
- Strategy
- State
- Command
- Iterator
- Mediator
- Template Method

Ventajas

- Mayor flexibilidad en comportamientos
- Mejor organización del código
- Facilitan mantenimiento y escalabilidad
- Reducen dependencias entre objetos
- Permiten reutilizar algoritmos y procesos

Desventajas

- Incrementan la cantidad de clases
- Pueden volver el sistema más complejo
- Algunos patrones son difíciles de implementar
- Mayor dificultad para depuración

¿Qué son los patrones de arquitectura?

Los patrones de arquitectura son soluciones generales y reutilizables que se utilizan para estructurar sistemas de software. Definen la organización de los componentes, la relación entre ellos y la forma en que interactúan dentro de una aplicación.

¿Para qué sirven?

- Organizan la estructura del sistema
- Facilitan el desarrollo y mantenimiento del software
- Mejoran la escalabilidad y reutilización
- Reducen la complejidad del sistema

Tipos de patrones de arquitectura

- Arquitectura en Capas (Layered Architecture)
- Cliente-Servidor
- MVC (Modelo Vista Controlador)
- Microservicios
- Arquitectura Hexagonal
- SOA (Arquitectura Orientada a Servicios)
- Event-Driven Architecture
- Pipe and Filter
- Monolítica

Ventajas

- Mejor organización del código
- Mayor mantenibilidad
- Facilitan escalabilidad y reutilización
- Reducen el acoplamiento entre componentes

Desventajas

- Pueden aumentar la complejidad inicial
- Requieren mayor planificación
- Algunos patrones son difíciles de implementar
- Incrementan la cantidad de componentes y clases

Justificación del uso de patrones

- **Bridge**

Se eligió porque el proyecto necesita separar dos elementos que cambian de forma independiente: las piezas o armas y los materiales. Este patrón evita crear una clase para cada combinación posible y facilita agregar nuevos materiales o piezas sin modificar las existentes.

- **Decorator**

Se eligió porque el equipamiento de Steve funciona de manera dinámica y acumulativa. Permite agregar o quitar piezas en cualquier momento, mientras los stats se actualizan automáticamente sin modificar la estructura principal del personaje.

- **Memento**

Se eligió para guardar y restaurar configuraciones completas del personaje. Gracias a este patrón, el usuario puede recuperar builds anteriores fácilmente sin tener que configurar nuevamente cada pieza o arma.

- **Factory Method**

Se eligió para centralizar la creación de objetos como piezas y materiales. De esta manera, la pantalla principal no necesita conocer las clases concretas, reduciendo el acoplamiento y facilitando agregar nuevos elementos al sistema.

- **MVVM**

Se eligió para separar la lógica de negocio de la interfaz gráfica. El ViewModel maneja el estado y la lógica del equipamiento, mientras que la vista solo se encarga de mostrar la información, haciendo el código más limpio y mantenible.

Compatibilidad entre los patrones

Relación entre Bridge y Decorator

Bridge y Decorator trabajan juntos dentro de la capa del modelo, pero cada uno tiene una función diferente. El Decorator se encarga de agregar las capas de equipamiento y sumar sus estadísticas, mientras que Bridge define el valor que aporta cada material. Ambos se complementan para calcular correctamente los atributos del personaje.

Relación entre Factory Method, Bridge y Decorator

Factory Method funciona como intermediario entre el ViewModel y los patrones Bridge y Decorator. El ViewModel solicita la creación de piezas y materiales, y la fábrica decide qué objetos instanciar. Esto reduce el acoplamiento y centraliza toda la lógica de creación en un solo lugar.

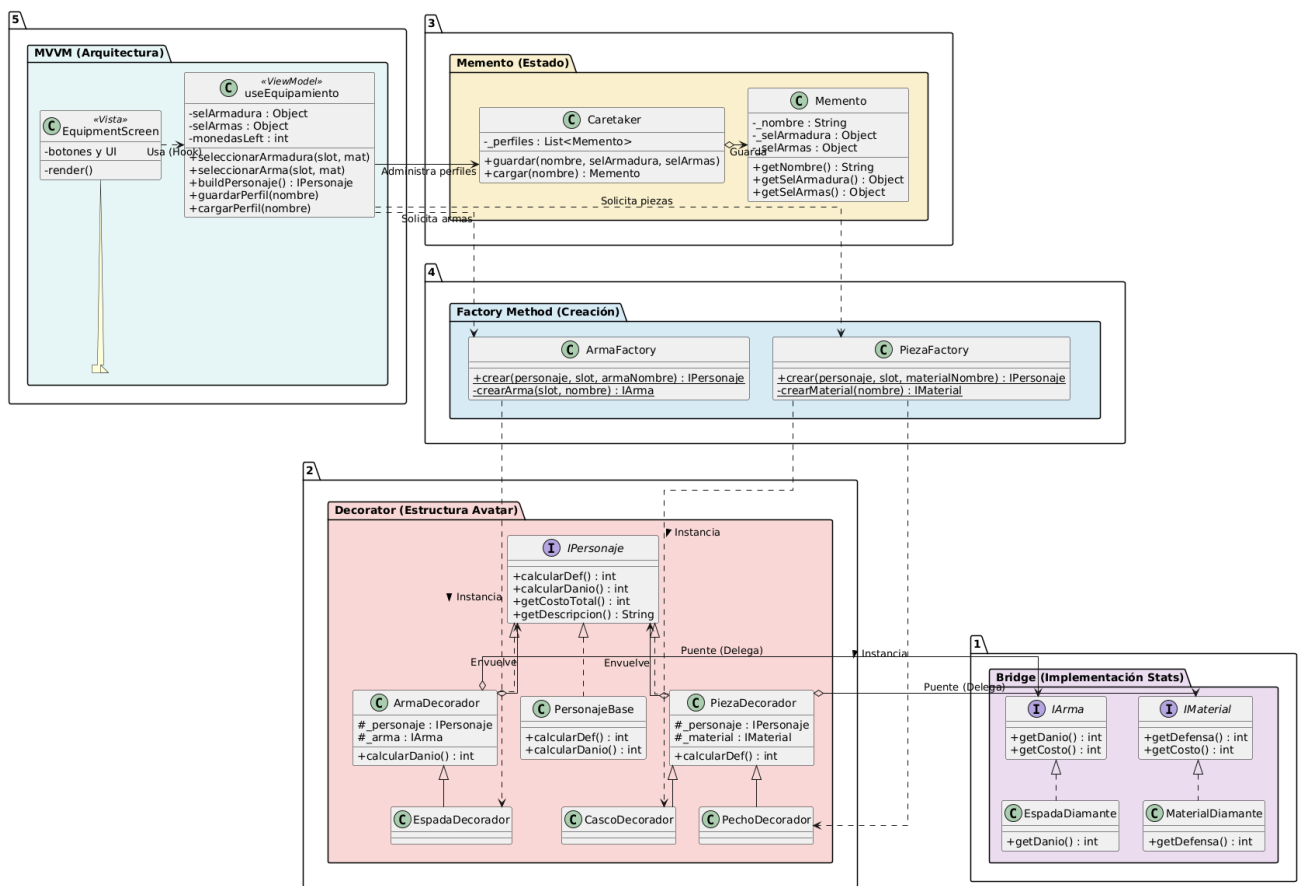
Relación entre Memento y los demás patrones

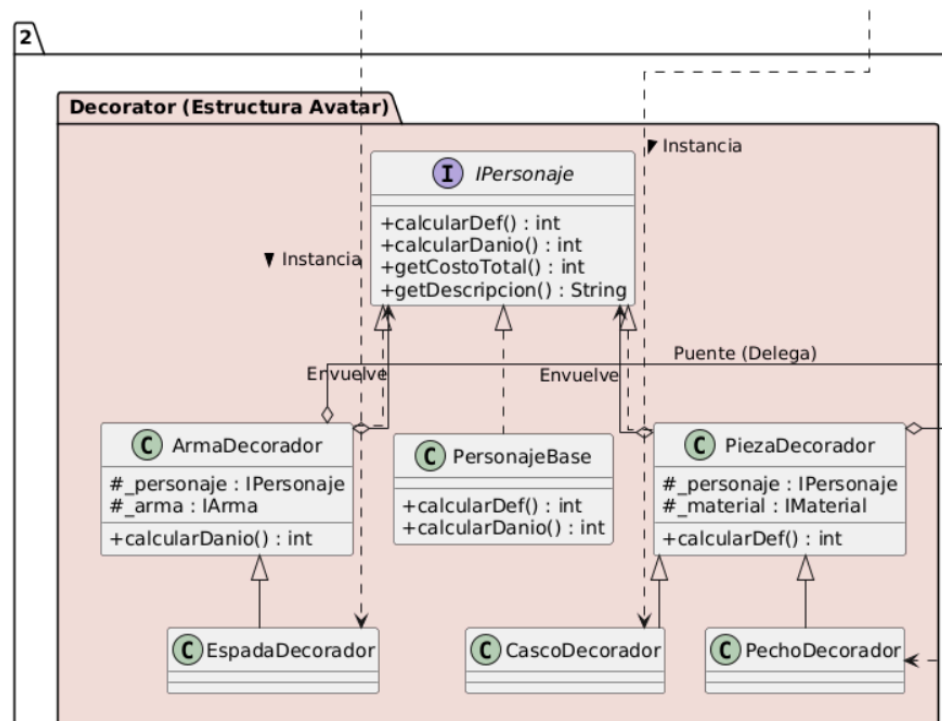
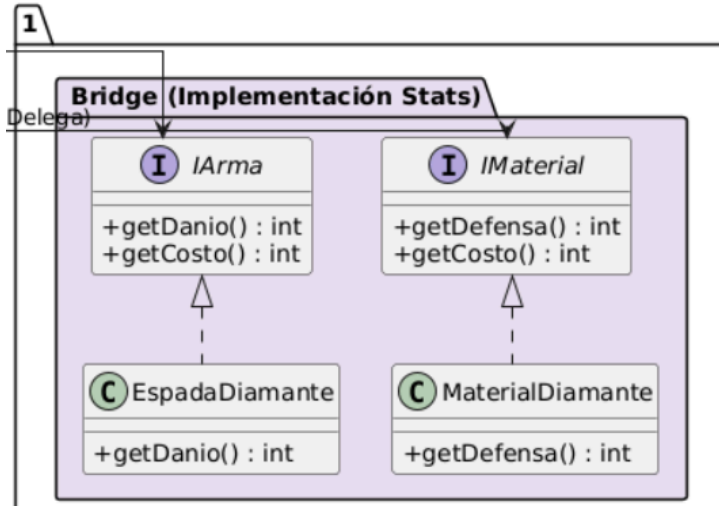
Memento trabaja de forma independiente del resto de patrones. Su única función es guardar y restaurar el estado del equipamiento mediante datos simples como nombres o selecciones. Después, el ViewModel utiliza Factory Method para reconstruir los objetos necesarios.

Relación entre MVVM y los demás patrones

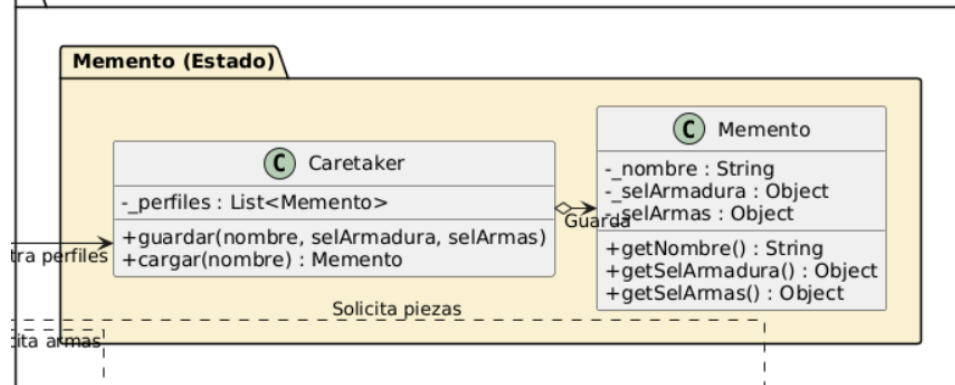
MVVM actúa como la arquitectura principal que organiza todos los patrones. El ViewModel coordina Factory Method, Memento, Bridge y Decorator, mientras que la Vista solo muestra la información procesada. Esto permite mantener separada la lógica de negocio de la interfaz gráfica y hace el sistema más limpio y mantenible.

Diagrama UML

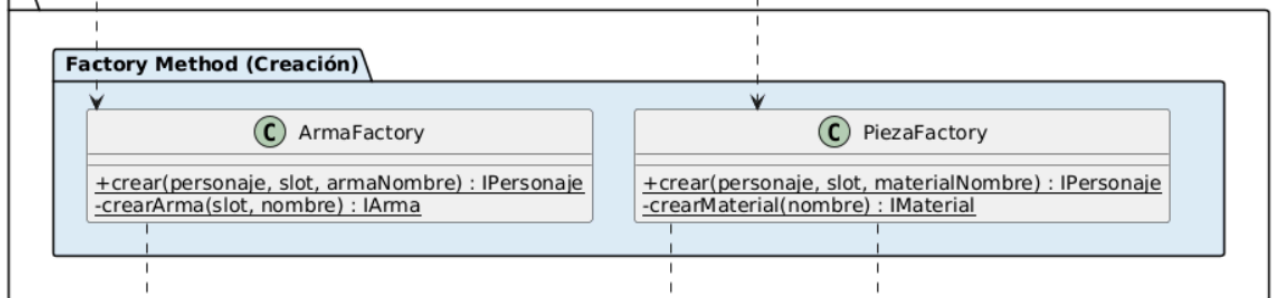




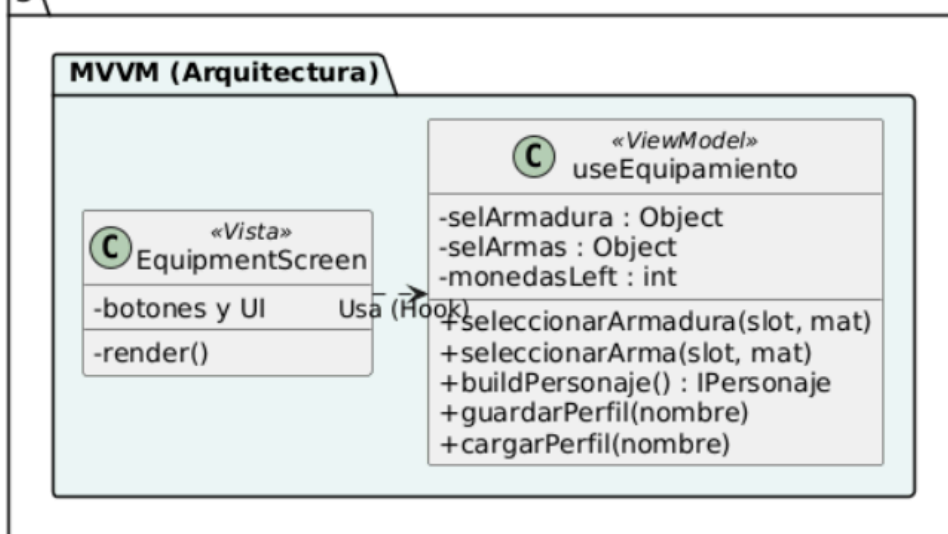
3



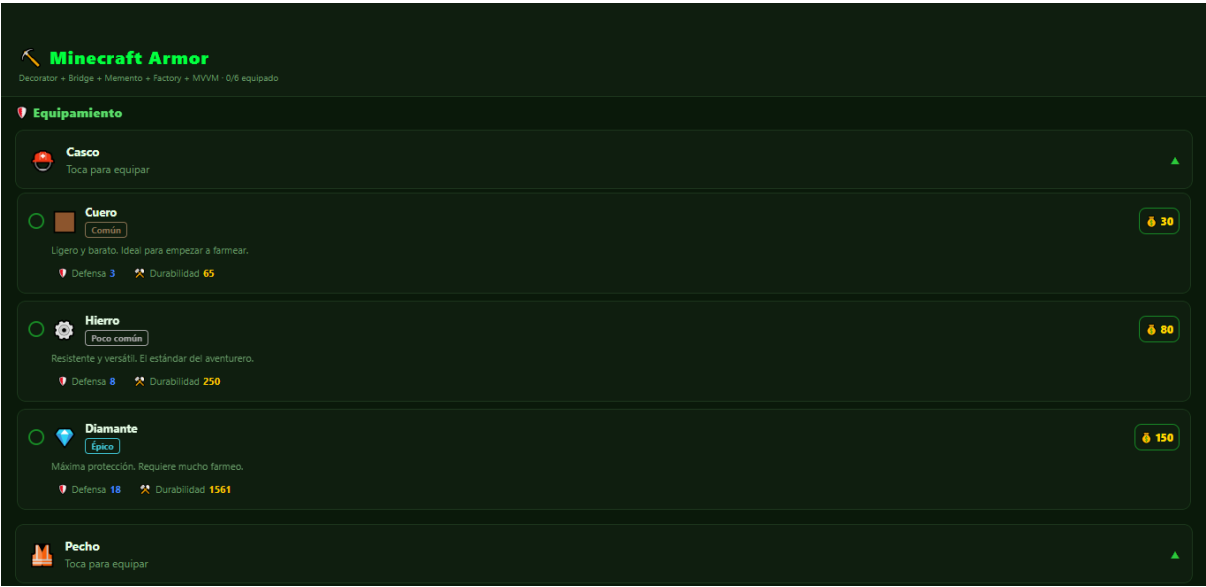
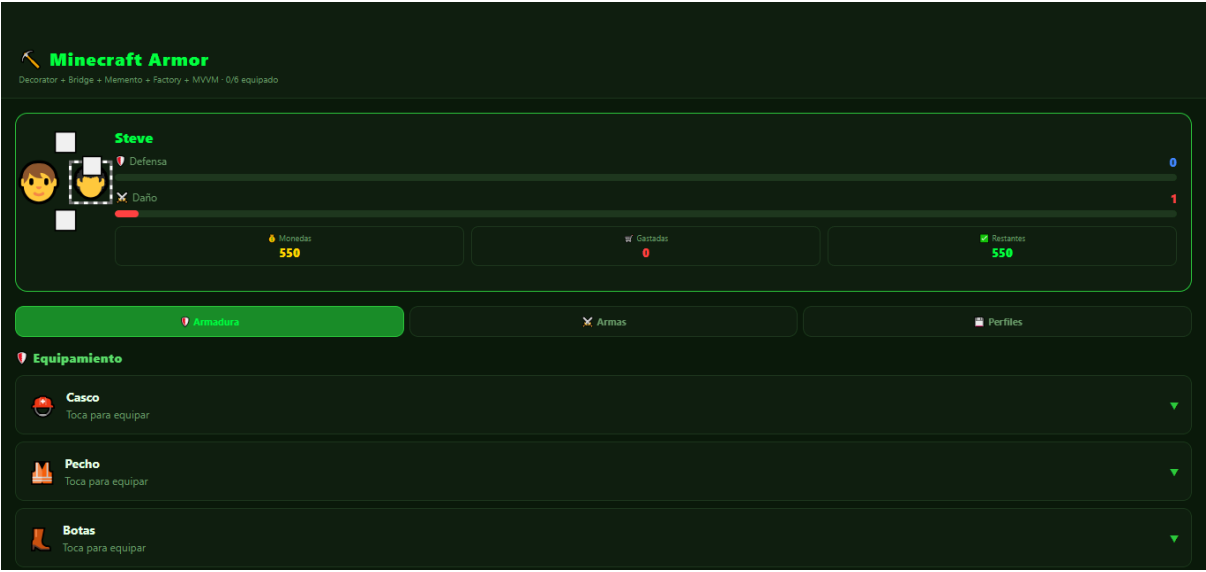
4



5



Capturas de programa



**Minecraft Armor**

Decorator + Bridge + Memento + Factory + MVVM - 1/6 equipado

Reset

Hacha

Ninguna

Madera
✕ 5 - 022

Hierro
✕ 9 - 080

Diamante
✕ 15 - 0150

Estadísticas de armas

Arco
Diamante

Épico

Daño

11

Durabilidad

1561 pts

Costo

145

Totales del personaje

Defensa total

0

Daño total

12

Durabilidad

1561 pts

Costo total

145

**Minecraft Armor**

Decorator + Bridge + Memento + Factory + MVVM - 5/6 equipado

Reset

Perfiles guardados

Nombre del perfil...

Guardar

Antonio

9:12:06 a.m.

Cargar

Hierro

Diamante

Hierro

Diamante

Hierro

Estadísticas de armadura

Casco
Hierro

Poco común

Defensa

8

Durabilidad

250 pts

Costo

80

Pecho
Diamante

Épico

Defensa

18

Durabilidad

1561 pts

Costo

150

Conclusión

- Los patrones de diseño ayudan a resolver problemas complejos de forma organizada y eficiente. En este proyecto, Bridge y Decorator evitaron la creación de demasiadas clases para cada combinación de piezas y materiales, haciendo el código más limpio, reutilizable y fácil de ampliar.
- La separación de responsabilidades mediante MVVM mejoró la estructura del proyecto. Al separar la lógica de la interfaz, el código se vuelve más claro, fácil de mantener y menos propenso a errores, además de facilitar el trabajo colaborativo entre desarrolladores.
- Los patrones de diseño son más efectivos cuando trabajan en conjunto. Cada patrón cumple una función específica y se conecta con los demás para formar un sistema más flexible, organizado y fácil de entender, donde cada componente tiene una responsabilidad clara.
- Una arquitectura basada en patrones permite que el proyecto crezca sin complicaciones. Agregar nuevas armas, materiales o funcionalidades requiere pocos cambios en el sistema, lo que facilita la escalabilidad y evita afectar el código ya existente.

Bibliografias

- Erich Gamma, Richard Helm, Ralph Johnson, & John Vlissides. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Robert C. Martin. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Martin Fowler. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Steve McConnell. (2004). *Code Complete*. Microsoft Press.
- [Refactoring Guru – Design Patterns](#)
- [Microsoft Learn – MVVM Pattern](#)
- Oracle Java Documentation – Design Patterns Overview